COMP219 Advanced Artificial Intelligence

Lab 6

Week 7

The Plan

Overview of Today's Lab

- Answer to last week's task
- Multilayer Perceptrons
  - Used in the assignment
- Assignment overview

Lab 5 Answers

Recap

- Topic: linear and logistic regression
- Task: model stealing
- Model stealing steps:
  1. Create noise and fit the victim model on the data
  2. Query the victim model on the noise
  3. Collect the output of the model
  4. Build our dataset
     - The noise is our training data
     - The victim model's output is our label/target
  5. Train our own model using the dataset from step 4.
  6. Metric: MSE between victim and stolen (surrogate) model
     - Aim: minimise MSE
     - Additionally, we can visualise the datasets and our model's decision boundary (or boundaries)

Multilayer Perceptrons

(Theory)

MLPs at a Glance

- Supervised learning
  - We split our data – training and test sets (common ratios are 80:20 and 70:30)
- Mainly used for classification but can be used for regression
- The MLP is first trained on a dataset then inferencing can be done with the trained model on unseen data

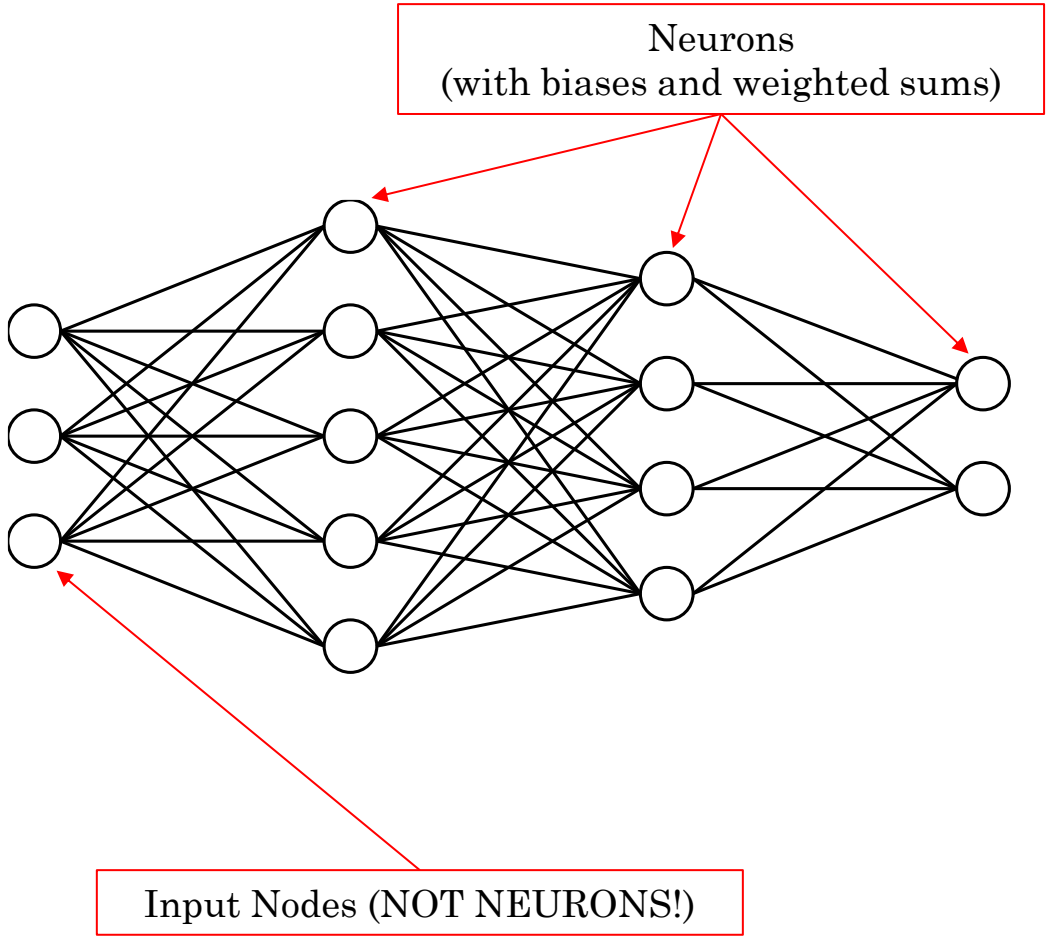
- Overview of the training process:
  1. Take a sample form the training data
  2. Process the input (convert it into a vector)
  3. Pass it through the neural network (the MLP)
  4. Calculate the error
  5. Feed the error back to adjust the parameters
  6. Repeat until convergence
- Overview of the inferencing process:
  1. Take the input (e.g. an image)
  2. Convert it into a vector
  3. Predict

The Anatomy of an MLP

- Fixed hyperparameters (set manually)
  - Input Layer
  - Hidden Layers
  - Output Layer
  - Neurons
  - Activation functions
  - Optimiser and regulariser
  - Epochs
  - Batch size
- Learnable parameters (some are randomised, some are set manually)
  - Weights (randomised, learned)
  - Learning rate (set manually, automatically adjusted over time)
  - Bias (randomised, learned)
  - Weighted sum (more on this later…)

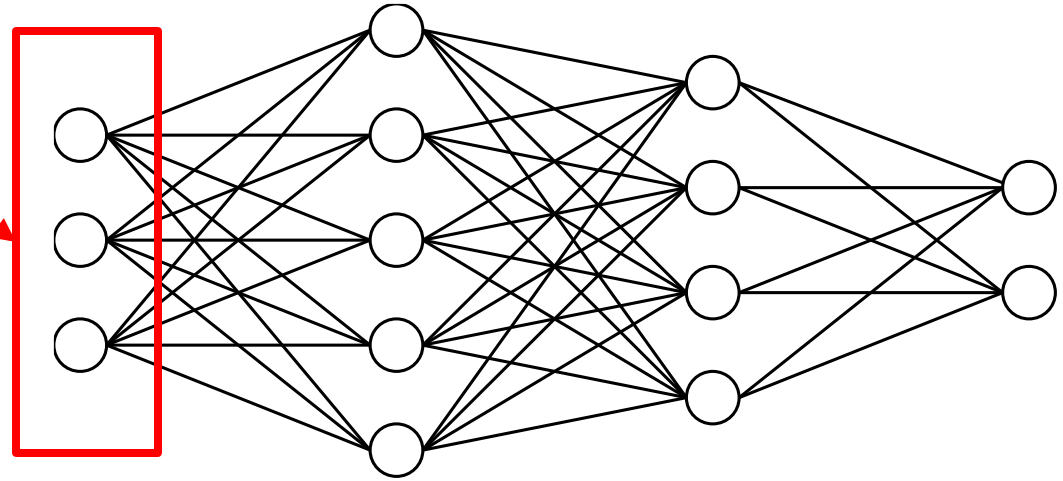
The Anatomy of an MLP: Layers

- Input Layer
  - Passes the processed input to the network
- Hidden Layer(s)
  - Performs calculations
- Output Layer
  - Performs the prediction
- Weighted connections
  - Specifies the signal strength
- Bias term
- Weighted sum
  - Each neuron has an output which is the weighted sum of the input to that neuron plus the bias fed through an activation function



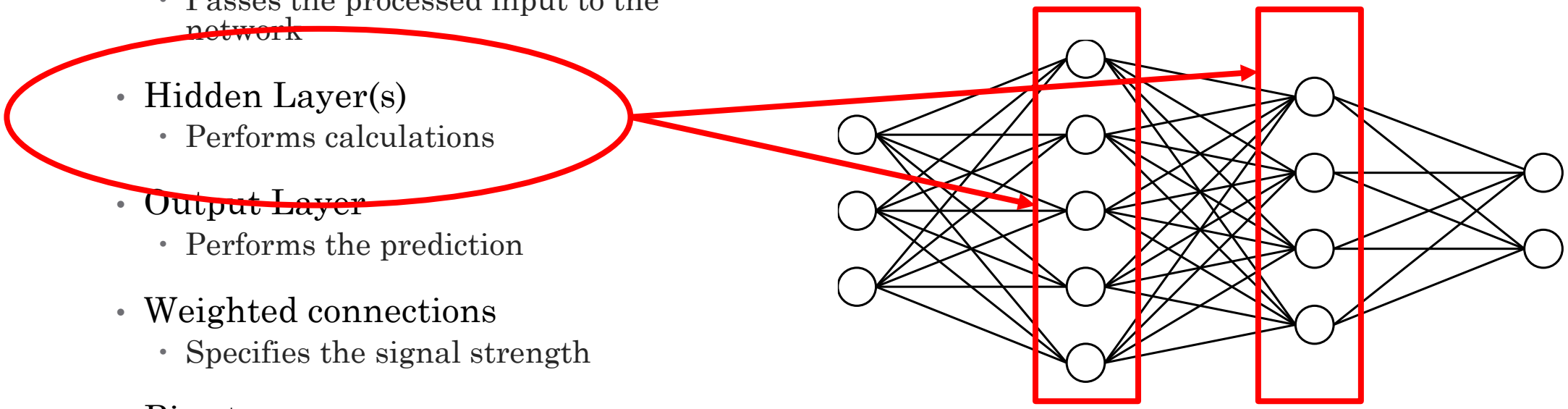
The Anatomy of an MLP: Layers

- Input Layer
  - Passes the processed input to the network
- Hidden Layer(s)
  - Performs calculations
- Output Layer
  - Performs the prediction
- Weighted connections
  - Specifies the signal strength
- Bias term
- Weighted sum
  - Each neuron has an output which is the weighted sum of the input to that neuron plus the bias fed through an activation function



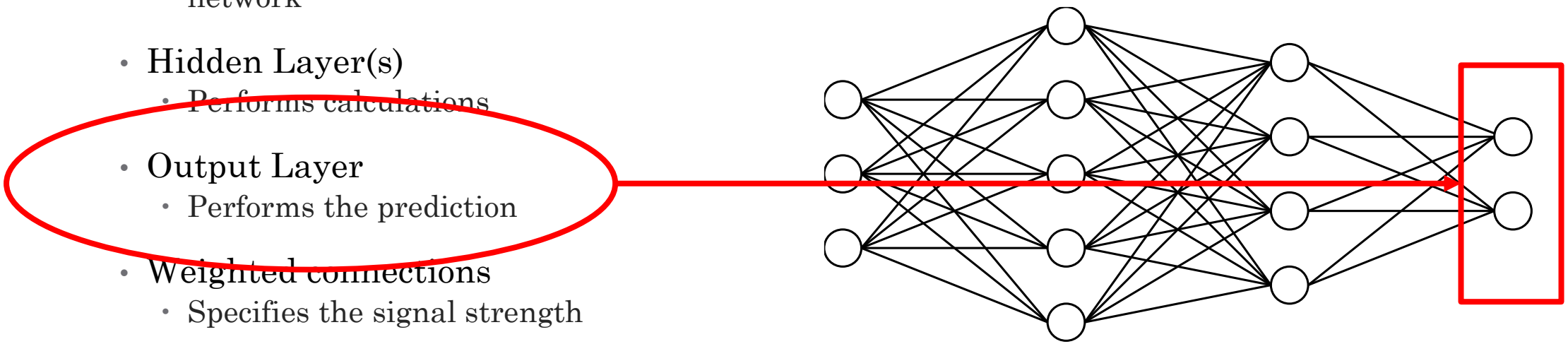
The Anatomy of an MLP: Layers

- Input Layer
  - Passes the processed input to the network
- Hidden Layer(s)
  - Performs calculations
- Output Layer
  - Performs the prediction
- Weighted connections
  - Specifies the signal strength
- Bias term
- Weighted sum
  - Each neuron has an output which is the weighted sum of the input to that neuron plus the bias fed through an activation function



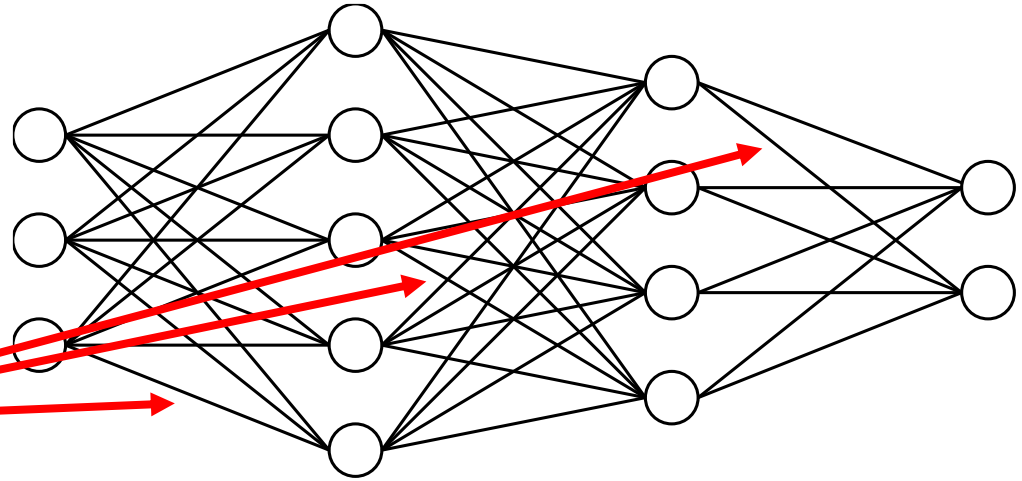
The Anatomy of an MLP: Layers

- Input Layer
  - Passes the processed input to the network
- Hidden Layer(s)
  - Performs calculations
- Output Layer
  - Performs the prediction
- Weighted connections
  - Specifies the signal strength
- Bias term
- Weighted sum
  - Each neuron has an output which is the weighted sum of the input to that neuron plus the bias fed through an activation function



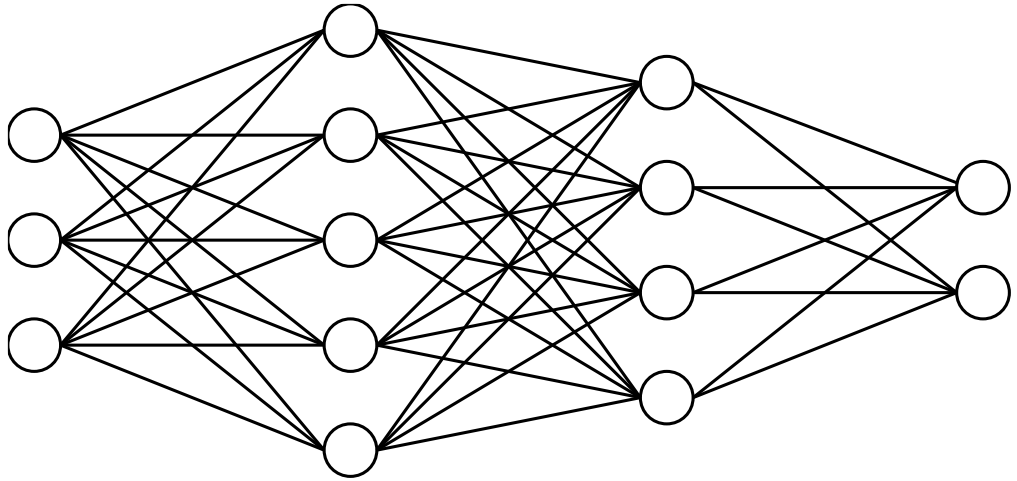
The Anatomy of an MLP: Layers

- Input Layer
  - Passes the processed input to the network
- Hidden Layer(s)
  - Performs calculations
- Output Layer
  - Performs the prediction
- Weighted connections
  - Specifies the signal strength
- Bias term
- Weighted sum
  - Each neuron has an output which is the weighted sum of the input to that neuron plus the bias fed through an activation function



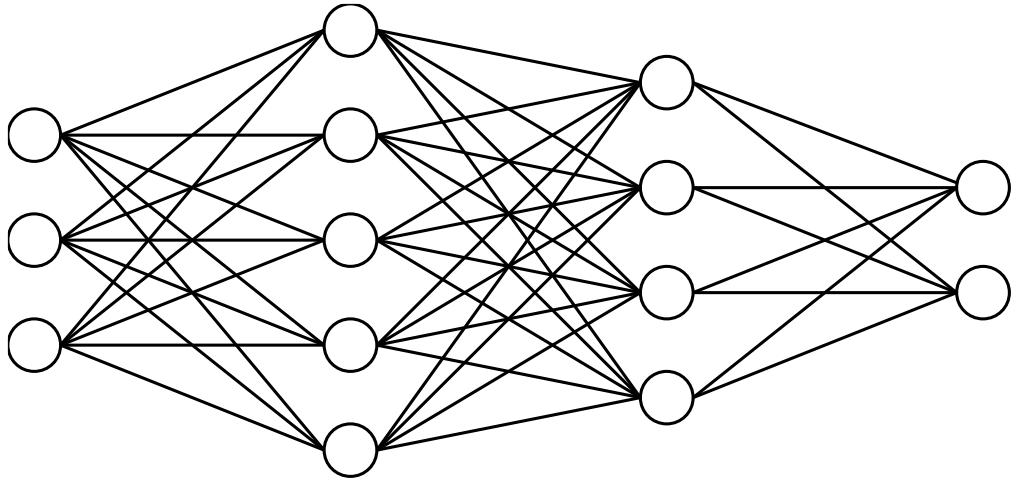
How Many Neurons and Layers?

- The number of input nodes is determined by the input data
  - For example, the images in the MNIST dataset are 28x28 pixels, so we would need 784 input nodes (one for each pixel)
- We determine the number of hidden neurons randomly
  - This number is usually influenced by the size of our data
  - Too many neurons and hidden layers will overfit
  - Too few will never converge
- The number of neurons in the output layer is the number of classes we have
  - For regression, we use a single neuron only



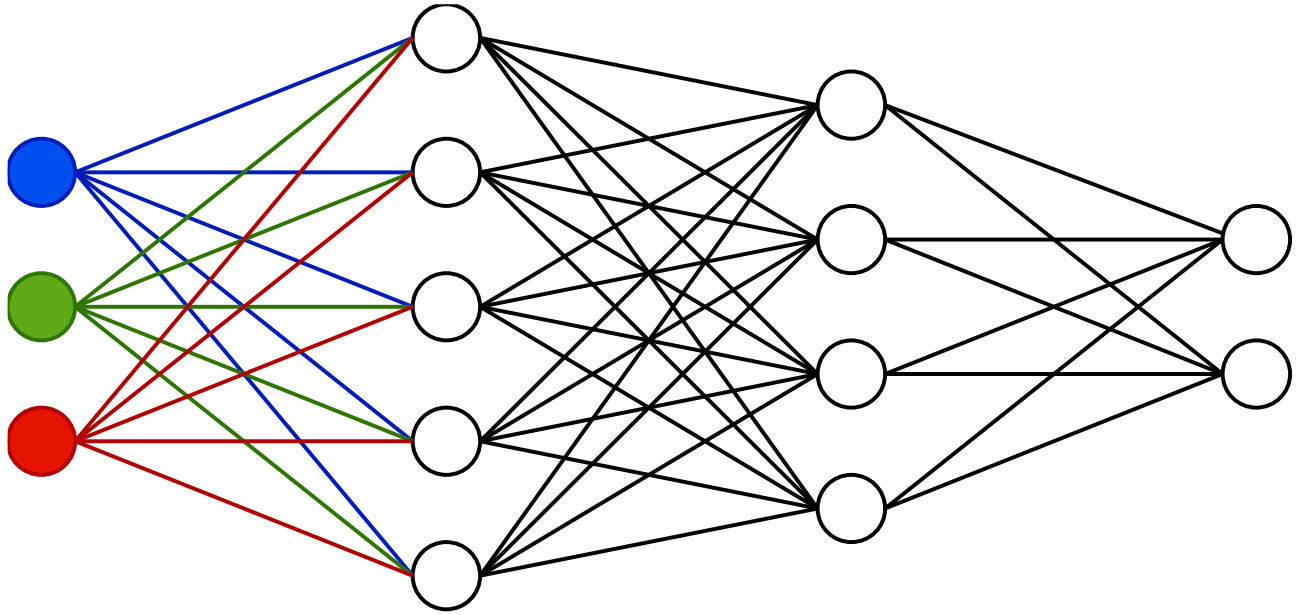
Feedforward and Backpropagation

- Information travels forward in the network **from the input layer through the hidden layers**, all the way to the **output layer**
- Each connection between the neurons is weighted which will produce varying signal strengths
- This results in one of the output neurons to fire (in classification)
- During training, we use backpropagation to train the model
  - We feed back the error so the model can learn from it



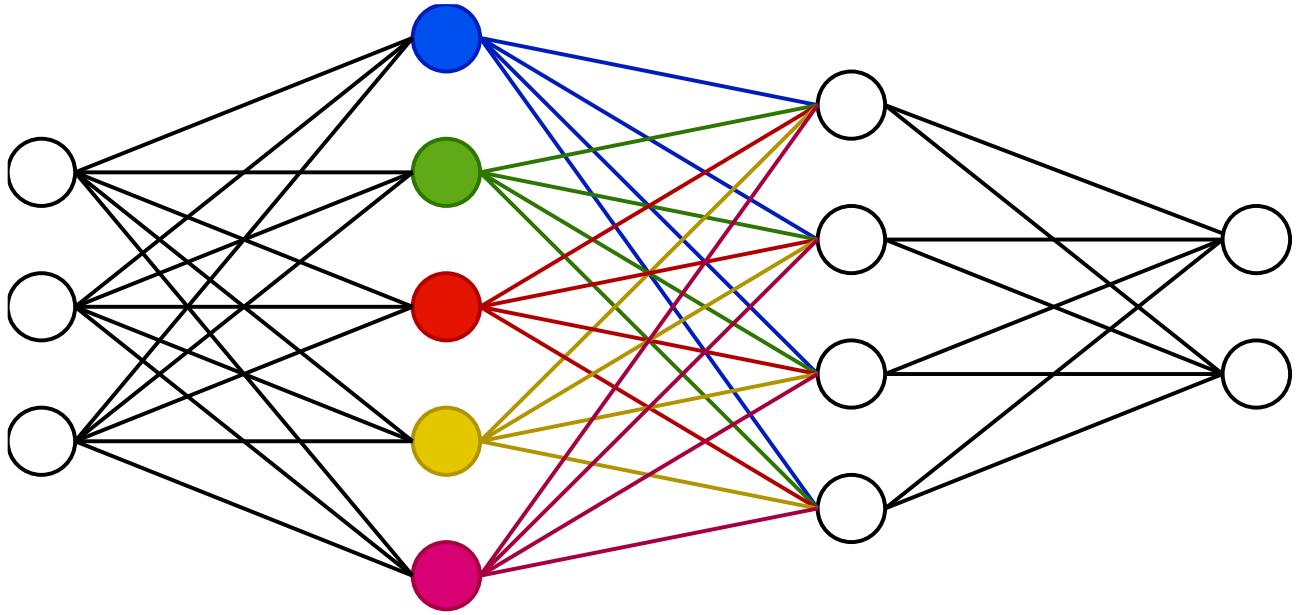
Feedforward and Backpropagation

- Start at the input layer
- Pass data to the neurons in the next layer via the connections
- Each node in an MLP is connected to the node in the next layer
  - “Interconnected”



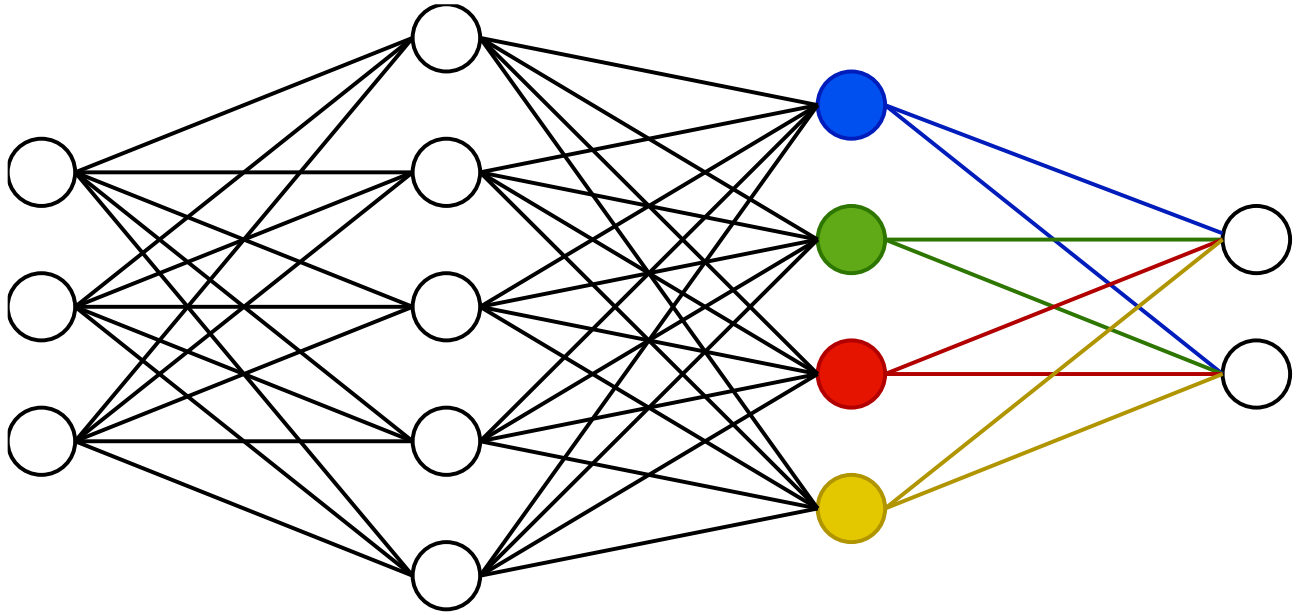
Feedforward and Backpropagation

- For each of the neurons in the hidden layer, we calculate:
  1. The weighted sum
  2. Add the bias to the weighted sum
  3. Pass the result through an activation function
  4. The result of the activation function becomes the output of that neuron
  5. Pass the output to the next neurons
  6. Repeat from Step 1 until the hidden layer is reached



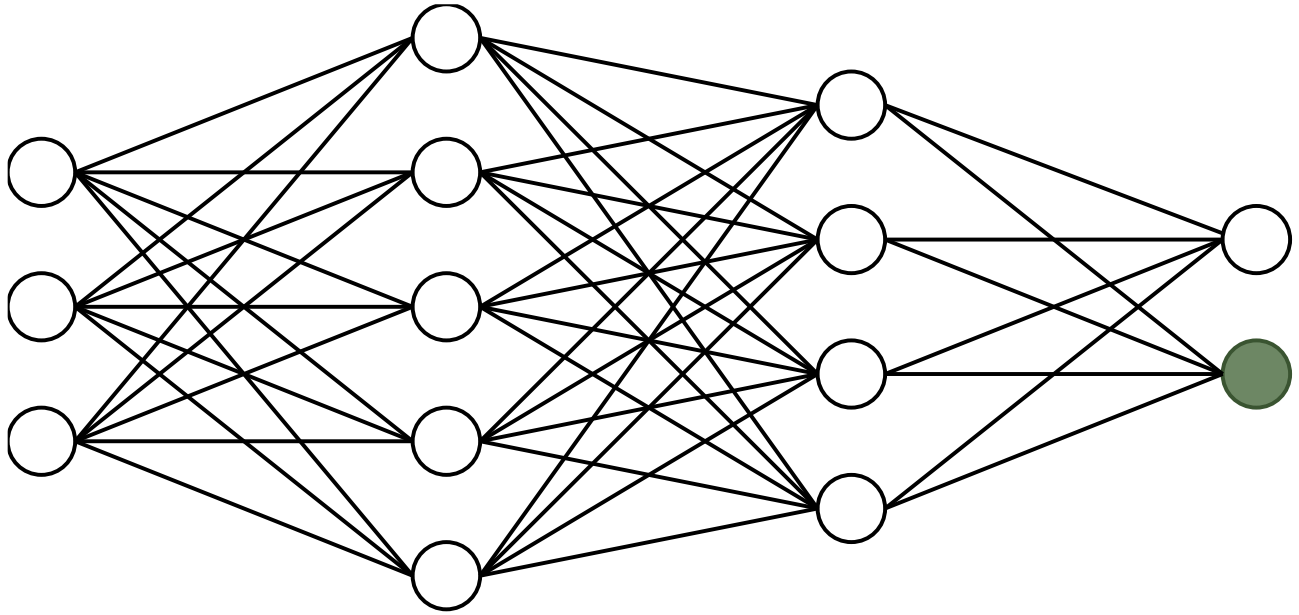
Feedforward and Backpropagation

- For each of the neurons in the hidden layer, we calculate:
  1. The weighted sum
  2. Add the bias to the weighted sum
  3. Pass the result through an activation function
  4. The result of the activation function becomes the output of that neuron
  5. Pass the output to the next neurons
  6. Repeat from Step 1 until the hidden layer is reached



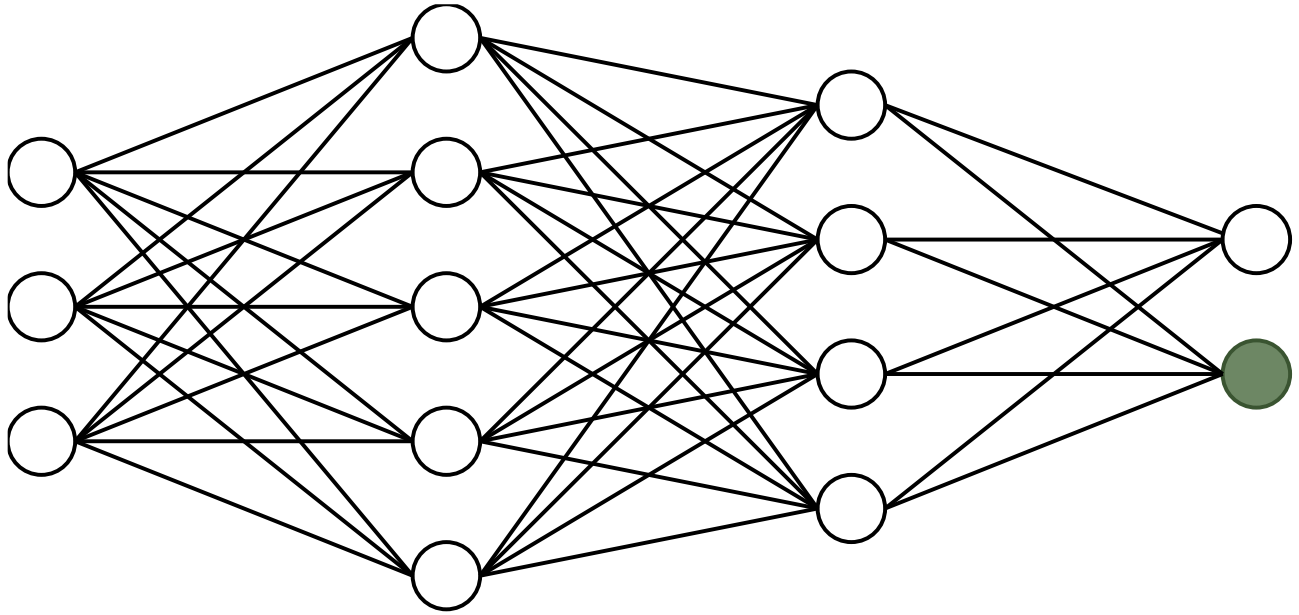
Feedforward and Backpropagation

- In the hidden layer:
  1. Weighted sum + bias
  2. Activation function
  3. Determine which neuron has the highest value
  4. That neuron wins, our prediction is the label associated with that neuron
- This is feedforward (or forward propagation), a technique by which MLPs predict
- To train the model, we send the error signal back all the way to the input layer to update weights and biases



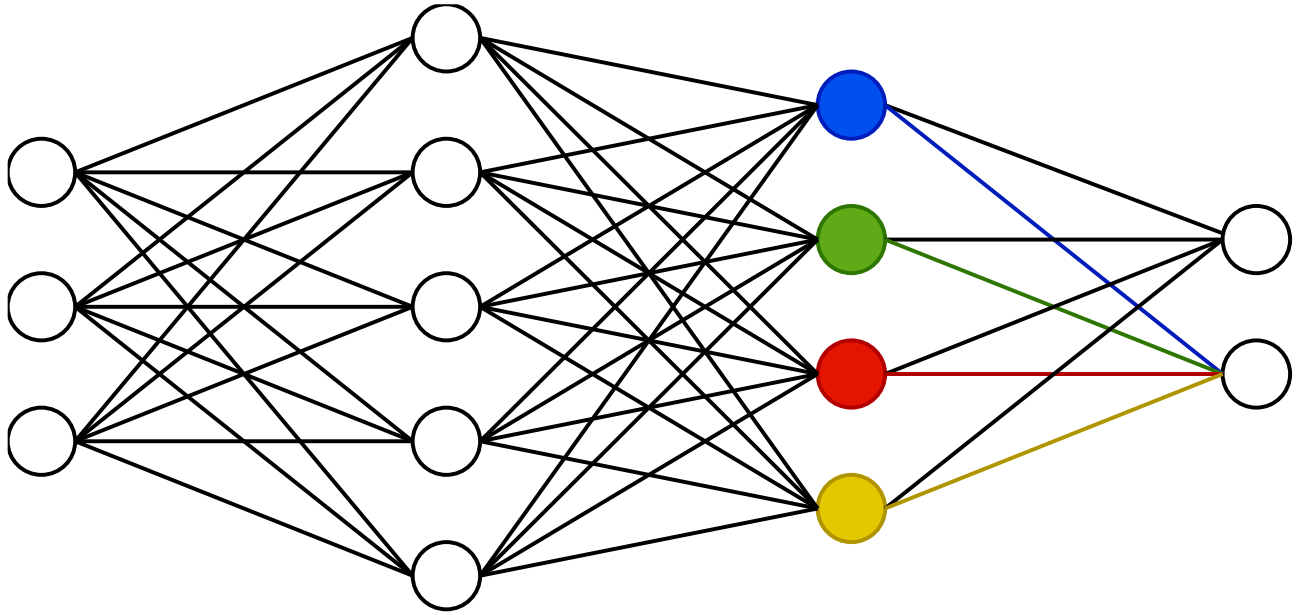
Feedforward and Backpropagation

- To update weights and biases, we need to:
  1. Calculate the error
  2. Find the partial derivative of the error
  3. Use stochastic gradient descent to update weights



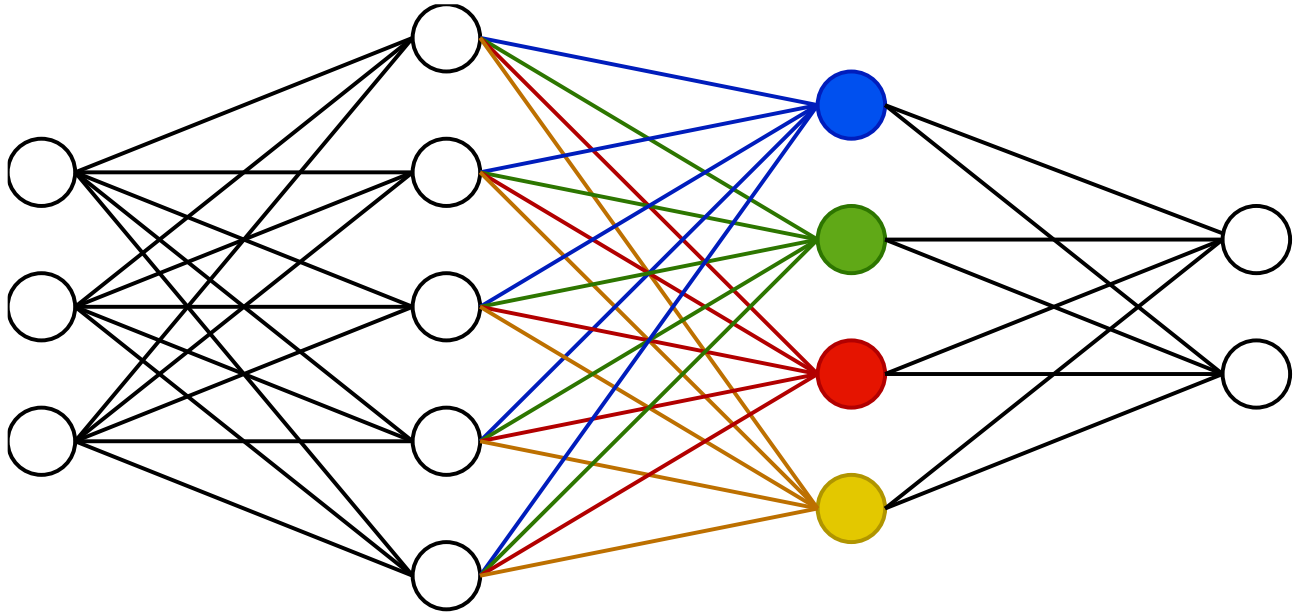
Feedforward and Backpropagation

- To update weights and biases, we need to:
  1. Calculate the error
  2. Find the partial derivative of the error
  3. Use stochastic gradient descent to update weights



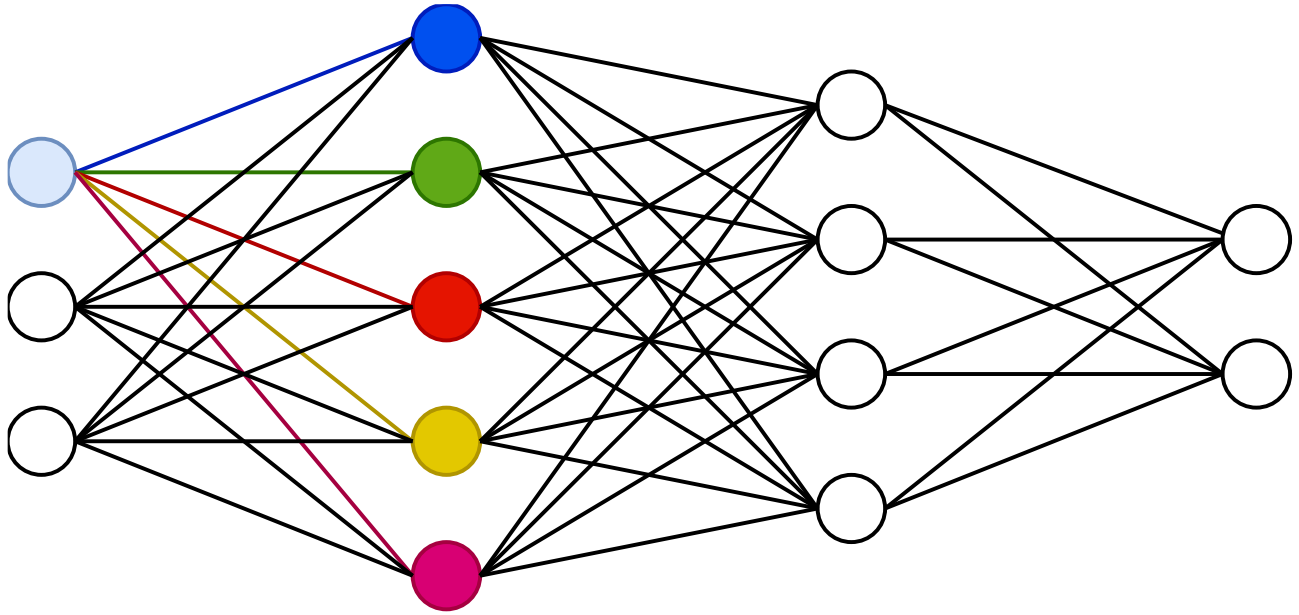
Feedforward and Backpropagation

- To update weights and biases, we need to:
  1. Calculate the error
  2. Find the partial derivative of the error
  3. Use stochastic gradient descent to update weights



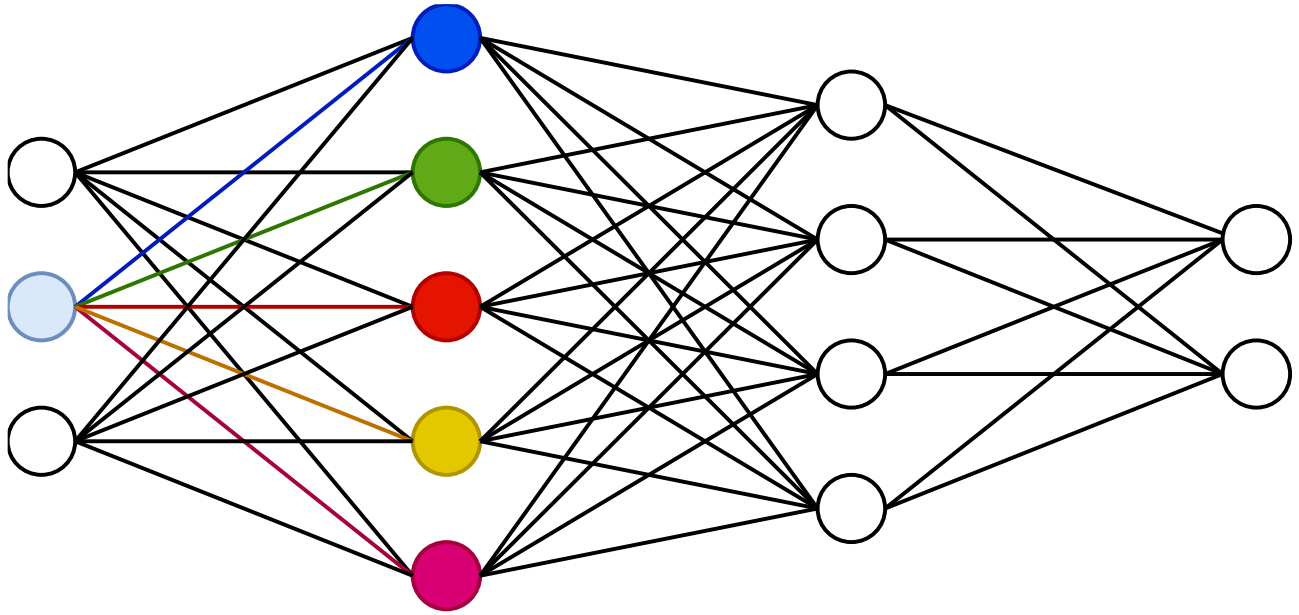
Feedforward and Backpropagation

- To update weights and biases, we need to:
  1. Calculate the error
  2. Find the partial derivative of the error
  3. Use stochastic gradient descent to update weights



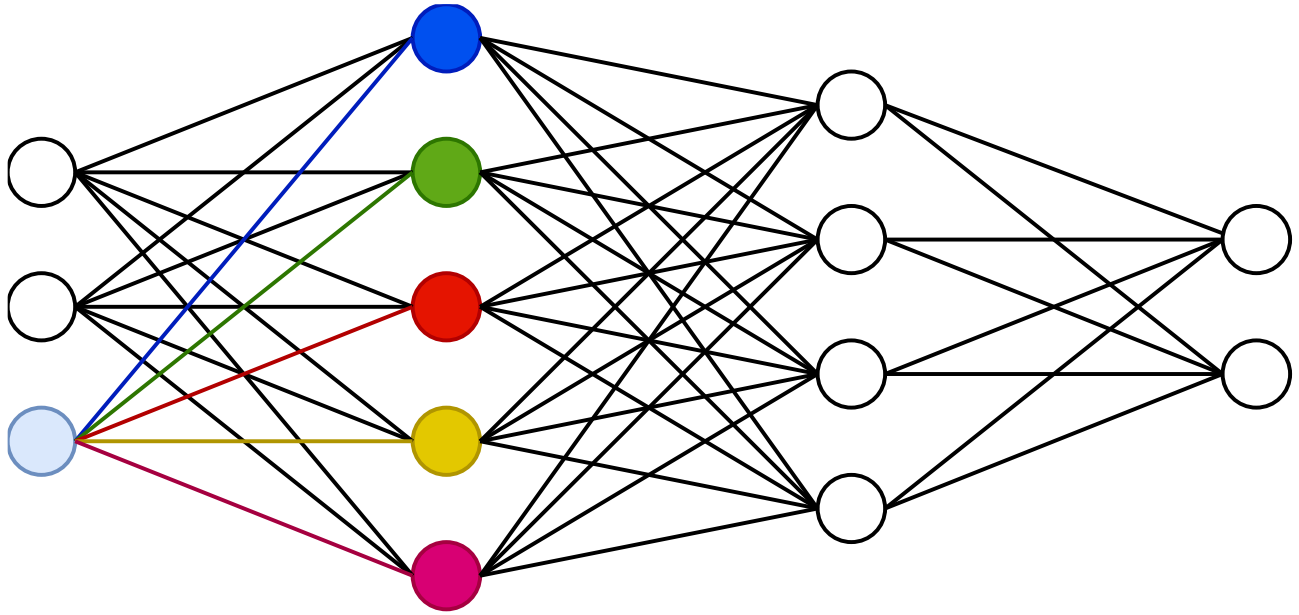
Feedforward and Backpropagation

- To update weights and biases, we need to:
  1. Calculate the error
  2. Find the partial derivative of the error
  3. Use stochastic gradient descent to update weights



Feedforward and Backpropagation

- To update weights and biases, we need to:
  1. Calculate the error
  2. Find the partial derivative of the error
  3. Use stochastic gradient descent to update weights
- The gradient of the loss with respect to each bias is calculated using the chain rule of calculus



Deep Dive

- Information travels forward in the network **from the input layer through the hidden layers**, all the way to the **output layer**
- Each connection between the neurons is weighted which will produce varying signal strengths
- This results in one of the output neurons to fire (in classification)
- During training, we use backpropagation to train the model
  - We feed back the error so the model can learn from it

The Maths

- Weighted sum

$$z = \sum_{i=1}^{n} w_i x_i + b$$

  - z is the weighted sum (pre-activation value) for the neuron
  - n is the number of neurons
  - w_i is the weight associated with the i th input
  - x_i is the i th input feature from the previous layer
  - b is the bias term for the neuron

- Stochastic Gradient Descent

$$w_i^{(t+i)} = w_i^t - \eta \cdot \frac{\partial L}{\partial w_i}$$

  - w_i^t is the current weight
  - $w_i^{(t+i)}$ is the updated weight
  - η is the learning rate
  - $\frac{\partial L}{\partial w_i}$ is the derivative of the loss function L with respect to the weight w_i

The Loss Function

- Measures how well the model does – the model's objective is to minimise the loss
  - The loss is calculated during training, when the model makes its prediction
  - The predicted value is compared against the actual value
- MSE or MAE for regression
- Cross-entropy for multiclass
- Binary cross-entropy for binary classification

- The loss function is not a hyperparameter!

Activation Functions

- For hidden layers: rectified linear unit (ReLU)

$$f(z) = max(0, z)$$

- For the output layer
  - Sigmoid (suitable for binary classification)
    - Suffers from vanishing gradient

$$f(z) = \frac{1}{1 + e^{-z}}$$

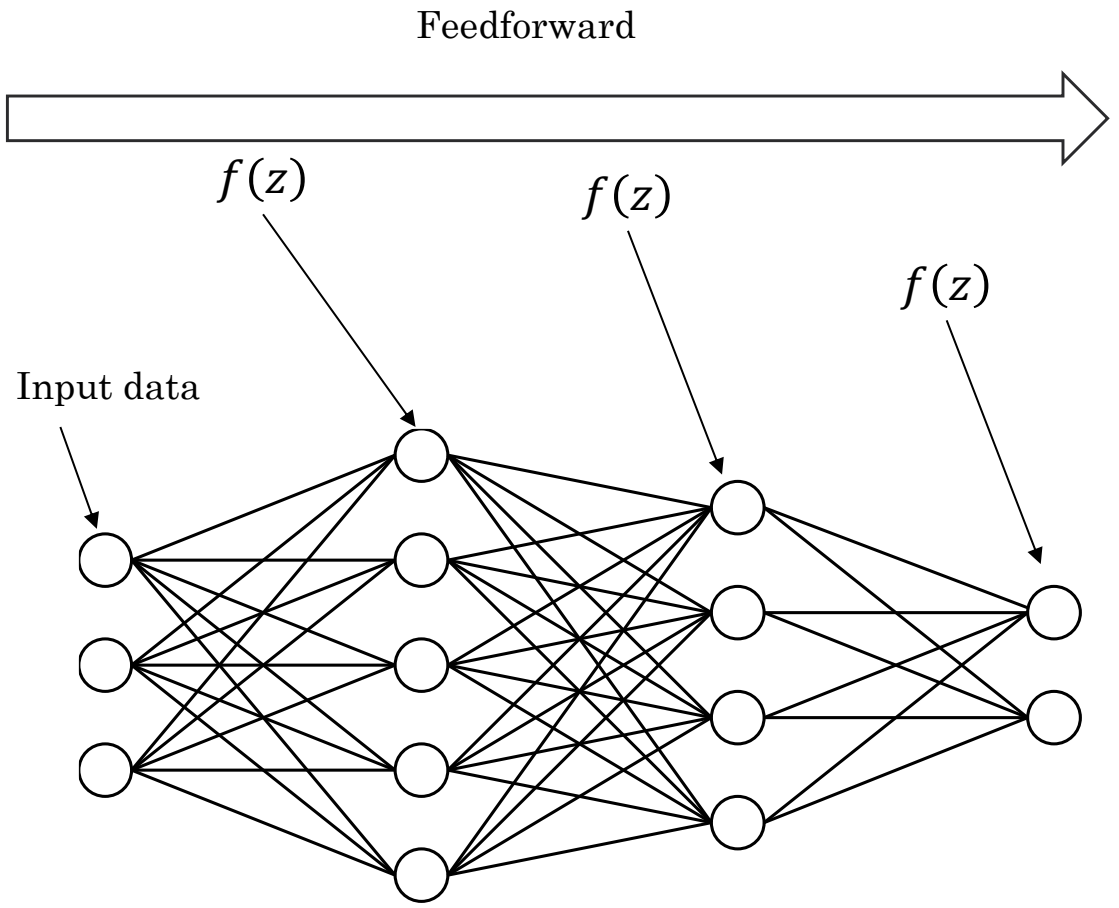
  - Softmax (suitable for multiclass)
    - Overconfidence in predictions
    - Problems classes are imbalanced

$$\sigma(z)_j = \frac{e^{x_j}}{\sum_{k=1}^{K} e^{x_k}} \; for \; j = 1, \ldots, K$$

Reminder: ***z*** is the weighted sum plus the bias of the neuron

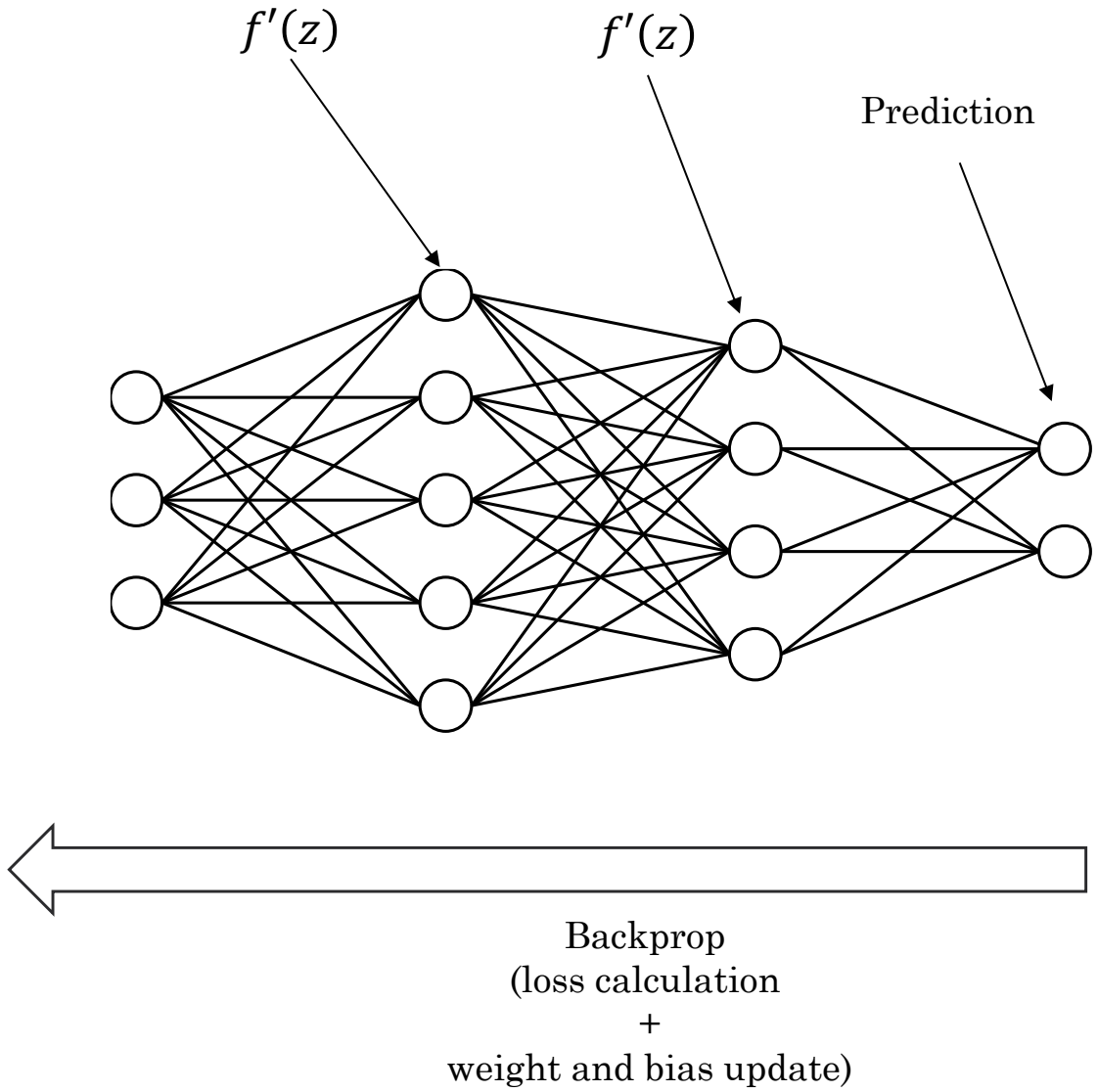
Summary

- Information travels through the network via the layers
- For each neuron:
  1. Calculate weighted sum
  2. Add bias
  3. Calculate the output by feeding the value into an activation function
- Prediction is made in the output layer
  - The neuron with the highest post-activation value predicts the class



Summary

- Information travels through the network via the layers
- For each neuron:
  1. Calculate weighted sum
  2. Add bias
  3. Calculate the output by feeding the value into an activation function
- Training:
  1. Calculate loss
  2. Get the gradient of the loss (the derivative of the error) for each weight and bias
  3. Update each weight and bias
  4. Repeat until a number of epochs (training loops)



Code Overview

Assignment Brief

General Information

- Go to the GitHub page
  - Go to competition
  - Check README_baseline.md file for guidance
- Careful: you are not allowed to change the MLP's architecture and model
  - The number of layers and neurons must remain the same
- Do not change the hyperparameters
  - It's not very helpful to change these parameters
- Do not change the datasets
- You will need to complete the functions:
  - *adv_attack (line91)*: you will implement your adversarial attack here
  - *train_model(line 165) train (line 108)*: define your adversarial training algorithm here
    - This function calls the *train* function, which is where you'll have your adversarial defence implemented
    - By default, the *train* function trains the model normally
  - *train (line 108)*: If needed, you could edit this function

Model evaluation

1. Download Files from GitHub

Download the following files from the GitHub repository:

- `Competition.py` - Your submission template
- `Reference_attacks.py` - Reference attack methods (for testing only)
- `Evaluate_attack.py` - Attack score evaluation script
- `Evaluate_defence.py` - Defence score evaluation script
- `README_baseline.md` - This guide

Optional (for Extended mode testing):

- `Extended_attacks.py` - Extended Attack loader
- `_attacks_internal/` folder - 15 compiled attack files (.pyc)

2. Download Reference Models from Canvas

1. Go to Canvas → COMP219 Module → Lecture 14
2. Download `Defenders.zip`
3. Extract to `./Defenders` folder in Competition directory

Model evaluation

File Structure

```
Competition/
├── Competition.py             # Your submission file
├── Reference_attacks.py       # Baseline attacks (for testing only)
├── Evaluate_defence.py        # Test defence score
├── Evaluate_attack.py         # Test attack score
├── extended_attacks.py        # Extended Attack loader (optional)
├── _attacks_internal/         # Compiled attack files (optional)
│   ├── attack_001.pyc
│   ├── attack_002.pyc
│   └── ... (15 attacks total)
└── Defenders/                 # Reference models (download from Canvas - see Setup)
    ├── 0.pt
    ├── 1.pt
    └── ... (10 models total)
```

Note:

- The `Defenders/` folder is **not included in GitHub**. You must download `Defenders.zip` from Canvas → COMP219 Module → Lecture 14, then extract it to the Competition directory.
- The `extended_attacks.py` and `_attacks_internal/` are **optional** and only needed if you want to use extended mode testing.

Attack Ability Evaluation

```
python Evaluate_attack.py
```

Configure in `Evaluate_attack.py` :

```
# Step 1: Set Defenders path
defenders_path = "./Defenders"

# Step 2: Define your attack (copy from Competition.py)
def your_attack(model, X, y, device):
    # Your attack implementation here
    return X_adv
```

Output:

```
ATTACK MATRIX
Model          Robust Accuracy      Attack Score (1/acc)
0.pt                 52.30%              1.91
1.pt                 48.15%              2.08
...
Mean                 49.82%              2.15

ATTACK SCORE
Your Attack Score: 2.1523

Note:
  - Higher score is better (stronger attack)
  - Final grading will normalize scores across ALL students
```

Defence Ability Evaluation

```
python Evaluate_defence.py
```

Configure in `Evaluate_defence.py` :

```
# Step 1: Specify your model file
your_model_file = "1000.pt"  # <-- CHANGE THIS to your model path

# Step 2: Select attack mode
attack_mode = 'baseline'  # <-- CHANGE THIS to 'baseline' or 'extended'
```

Attack Modes

Baseline Mode - Fast testing (recommended for development)

- Tests against 3 reference attacks: FGSM, PGD-5, PGD-20
- Quick execution
- Good for rapid iteration during development

Extended Mode - Comprehensive testing (recommended before submission)

- Tests against 15 Advanced Attacks
- More diverse attack strategies
- Better assessment of defence robustness

Which mode to use?

- Use `'baseline'` for quick testing during development
- Use `'extended'` for thorough testing before final submission
- Your defence should perform well on both modes

Output Example:

Defence Ability Evaluation

Output Example:

```
DEFENCE EVALUATION RESULTS

Attack               Robust Accuracy
--------------------------------------------------------------------------------
FGSM                  45.23%
PGD-5                 38.67%
PGD-20                32.15%
--------------------------------------------------------------------------------
Mean (Defence Score)  38.68%

Your Defence Score: 0.3868

Note:
  - Higher score is better (more robust model)
  - Final grading will normalize scores across ALL students
  - Current mode: baseline
  - Tip: Try 'extended' mode for more comprehensive testing
```

Defence Ability Evaluation

Optional: Add Custom Attacks

You can add your own attacks for testing (Step 3 in `Evaluate_defence.py`):

```
def my_custom_attack(model, X, y, device):
    """
    Your custom attack implementation

    Args:
        model: the neural network to attack
        X: input data [batch_size, 784]
        y: true labels [batch_size]
        device: torch.device

    Returns:
        X_adv: adversarial examples [batch_size, 784]

    Note: Perturbation L∞ norm must be STRICTLY < 0.11 (use ≤ 0.109 to be safe)
    """
    epsilon = 0.1
    # Your attack logic here
    X_adv = X + epsilon * torch.randn_like(X)
    X_adv = torch.clamp(X_adv, 0.0, 1.0)
    return X_adv

# Add to test suite
test_attacks.append(('MyAttack', my_custom_attack))
```

Understanding Scores

Understanding Scores

Attack Score

- Calculated as: mean of `1/robust_accuracy` of your attack against reference models
- **Higher is better** (stronger attack)
- Lower robust accuracy → stronger attack → higher score

Defence Score

- Calculated as: mean robust accuracy of your model against reference attacks
- **Higher is better** (more robust model)

Final Grading

- Your self-test score is for reference only
- Final grading will normalize scores using **all students' scores**
- This ensures fair comparison across the class

Reference Attacks

Three baseline attacks are provided **for testing purposes only**:

Attack	Description
FGSM	Fast one-step attack
PGD-5	5-step iterative attack
PGD-20	20-step iterative attack

⚠️ IMPORTANT - Epsilon Constraint

- All attacks must satisfy: **L^∞ norm STRICTLY < 0.11** (NOT equal to 0.11)
- Safe value: use epsilon ≤ 0.109
- Verify your attack: check `p_distance` output in `Competition.py`

⚠️ IMPORTANT - Academic Integrity

- These baselines are **for evaluating your defence only**
- **DO NOT directly copy these methods for your submission**
- Simply using these attacks **will NOT give you good marks**
- You must **develop your own attack methods** based on:
  - Course materials (Lab 7.1 and 7.2)
  - Lecture content
  - Your own research and improvements

Develop your own methods! These baselines help you test, not solve the assignment.

Recommended Workflow

1. **Develop** your attack/defence in `Competition.py`
2. **Train** your model and generate `.pt` file using `Competition.py`
3. **Test** attack: ▷ `python Evaluate_attack.py`
4. **Test** defence (quick): Set `attack_mode = 'baseline'` in `Evaluate_defence.py` , then run ▷ `python Evaluate_defence.py`
5. **Test** defence (thorough): Set `attack_mode = 'extended'` in `Evaluate_defence.py` , then run ▷ `python Evaluate_defence.py`
6. **Iterate** based on scores
7. **Verify** epsilon constraint: check `p_distance < 0.11` in `Competition.py` output
8. **Submit** your final `Competition.py` + model `.pt` file